Kantonsschule im Lee Winterthur
Maturitätsarbeit HS 2025/26

Developing a playable simulation of markets and corporations

Attila Pinter, 4a
Patrick Hnilicka

December 29, 2025

Contents

1 Motivation

During Economy Week (Deutsch: Wirtschaftwoche), as an exercise in *business administration*, we *played* a game, where you were the board of a company manufacturing some arbitrary product and had to make decisions to increase sales, lower cost and generally beat out your competition. For me, the aspect of being able to come up with novel management ideas and compete with your friends really hooked me into the concept. This may sound hyperbolic, but during that week I genuinely woke up exited to go to school, not to actually learn stuff (and whatever) but just to play a few more rounds and (aspirationally) completely destroy my competition.

The game we got to play during Economy Week, in my opinion, had a tantalising hook but the interface was utterly hostile to non-directed interaction: it was pretty much an excel table! Anyway, I set off to build a better, deeper version of this simulation.

> "We do this not because it is easy, but because we thought it would be easy"
>
> **Not JFK.**

This is the running motto of this project.

Reflecting the main intention of this project, the name of the game is (currently) **Wisim**, from this point onwards all mentions of "Wisim" refer to the product of this project.

2 Concept

The goal of this project is to develop a game about leading a company in a competetive, multiplayer market. The game would have a dynamic market which has emergent properties resembling those of real economic models. Plyer would be able to compare their companies and see which ones are the most profiable, valuable, have the most market share, etc. and attempt to beat each other to be the most valuable company. It would be almost like Civilisation except that instead of leading a civilisation from the dawn of mankind to hundreds of years in the future you lead a company to the next quarter.

2.1 Core Gameplay Loop

The game would have 2 repeating phases: the decisions phase and the simulation phase. During the decisions phase each player would be able to make decisions

that would lead their company to riches or to ruin. These decisions would include but not be limited to:

- Creating various different products targeting specific sections of the market
- Controlling the promotion of said products, especially which aespects of the product they promote
- Setting up the infrastructure to produce the products
- Management of personelle.

During the simulation phase there would be no input by the player, instead the game would simulate the results of the decisions made in the previous phase and compile some reports allowing the players to understand what went well or poorly such that they can improve their decisions in the next step.

3 Design

Designing a user interface for such a game is not the simplest of endeavours because, unlike most games where the UI is an afterthought, in Wisim the user interface is a make or break factor for the game. As with many simulation games where the game acts as interface with which the player interacts with an underlying simulation like Hearts of Iron 4 or OpenTTD, the interface has to turn the abstract parameters and numbers of the simulation into concrete, understandable information that the player would understand in the context.

3.1 Design Philosophy and Layout

The most fundamental question in designing the game is which philosophy and layout to choose. Should I use a tabular layout and simply lay the data out in an almost 1:1 connection to the simulation or should there be a sort of dashboard representing what an executive might get on their desk or should the game's UI visually show the physical conditions of an enterprise?

Answering this question is tough and took significant time, however there are some options that could be ruled out quickly:

- A physical representation of the enterprise that shows how machines, offices, etc would exist physically →Such a design would have a scope far outside of what is reasonable within half a year.
- A simple tabular layout that resembles how the data is represented in the back-end →My judgment tells me that a design like that would make the

game rather boring and unintuitive. The values displayed would be too abstracted from what they represent and would just lead to confusion.

3.1.1 Attempt #1: Tabular Dashboard

The first attempt at a design was made in made in April 2025 with a tabular layout with hints of a more advanced dashboard. It was very helpful for testing out various aspects of the simulation, however it proved simply too abstract to be immersive all the while many operations were painfully slow and some mechanics could not reasonably be implemented while following the existing designs (such as individual management of personnel and machines).

Concretely I didn't find it fun to play and think few other people would have found it fun.

Figure 1: The first UI I made for the game

3.1.2 Attempt #2: Desktop Management system

The second attempt came a few weeks later after I saw a video on Youtube where someone was developing their own economic simulation [1] (a different aspect of

[1] I Programmed an Economy Simulator | conaticus | https://youtu.be/BBIpAyUZq5U?si=H2j5L-wdBHYlHaUZ

the economy). They used multiple windows on an in game desktop to represent the various actions you can make. This system allows for a lot more flexibility and more conducive to gameplay. From this point onwards I would generally follow this layout, however in different configurations.

This attempt would ultimately fail due to technical difficulties (further described in "Technology" chapter).

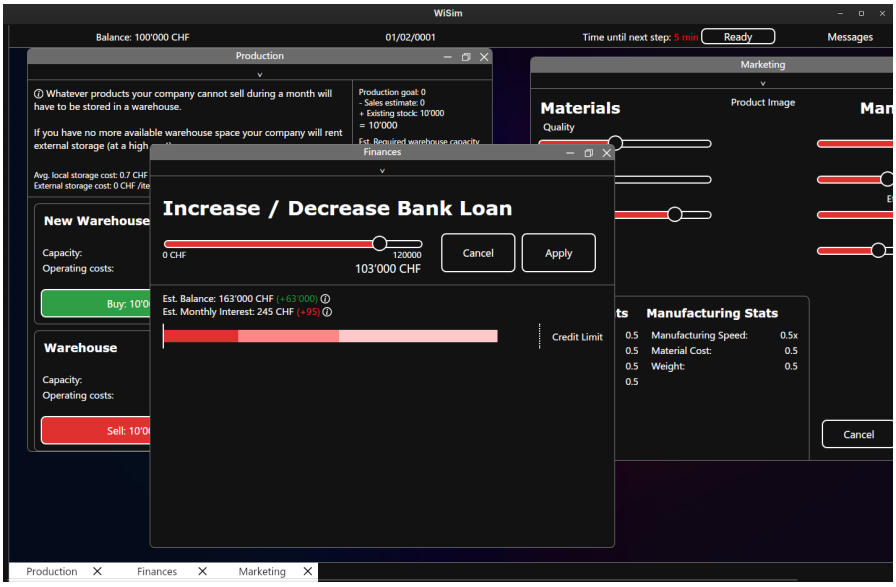

Figure 2: The second attempt at the User Interface

3.1.3 Attempt #3: Desktop Management system Godot

Due to the technical difficulties I experienced in the previous attempt, I tried switching to Godot due to the technical frustrations of the previous attempt. This had similar, but different technical problems and came to an end not very long after it started.

3.1.4 Attempt #4: Desktop Management: Finale

Finally I implemented a simplified version of the second attempt, this time with many technical and architectural improvements. Taking inspiration from games like Rimworld, Hearts of Iron 4 and Transport Fever 2, I added more graphics to the game to help the game feel more concrete and less theoretical. This version of the game, also included the product designer, which forced me to decide which

product to produce (coffee machines). This version would receive the most continuous investment from me and in my opinion was the most successful of the four.

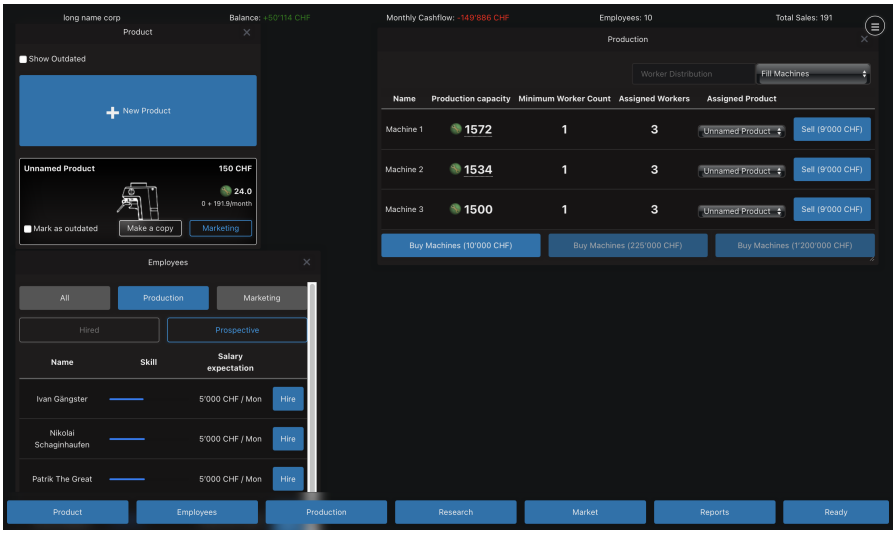

Figure 3: The final version of the user interface

3.2 Key Design Elements

In this section I would like to showcase certain elements of the game which took more time and were more complex to develop than other parts of the game.

3.2.1 Product Designer

The final version of the product designer was one of the most time consuming design elements of the whole game, as it not only required more complex code but also required rather deep changes to the game.

In early versions of the game, the product designer was less of a product *designer* and more of a product *configurator*; the product designer was a collection of sliders, each of which would modify a certain aspect or multiple assets of the product. This system had multiple flaws such as:

- Being unintuitive and intimidating to new players as it just hit the player with a wall of sliders
- Being kind of boing, since you were just modifying sliders and it didn't feel like you were truly designing your own product

- Being hard to balance; balancing the effects of each slider involved complex math with various complex curves not cause some fixed combination to become over powered.

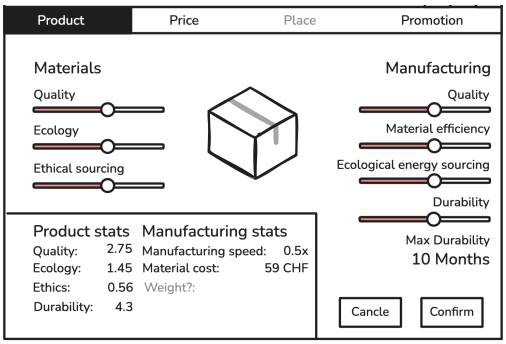

Figure 4: The original product designer

Thinking of how to fix the above problems, I remembered Hearts of Iron 4 (HOI4). HOI4 is a World War 2 simulation game and it suffers from the same problem of making an Excel table fun to play. One of their early updates included a new tank designer in which the player chose specific components to put in their tank. This system decreases the abstractness of the content while still allowing for lots of creativity. Inspired by HOI4 I also added a similar designer system, except this time for your companies products.

Implementing the new product designer did require some major changes to the game's scope: the most important of which being that I needed to choose which product would be produced (instead of a generic product), otherwise the whole product designer wouldn't make any sense. And so after (very short) deliberation, I decided that companies would produce coffee machines, for no deeper reason than there was a coffee machine sitting right in front of me.

Figure 5: Left: tank designer from HOI4, right: new product designer from Wisim

3.2.2 Prediction mode

One of the fundamental problems I and other early play testers experienced when testing early versions of the game was that it was extremely difficult to estimate how much money they would make with a certain set of decisions. They would either just take a blind guess as to whether they would make money or have to pull out a calculator and a notebook. The game made it very difficult to find out if your products were priced well or poorly and how decisions would impact the long term.

To solve this problem, I added "budget mode". When you hover your mouse over the "Ready" button you get the option to set a sales target for each product, then you can press the "Enter Budget Mode" button. When you pressed the button the colour scheme of the game would change and in the "Reports" window and at the top of the screen you would be able to see the *predicted* results of your decisions (assuming everything went according to plan). This system is probably the most useful tool for setting your prices, because it allows the player to make sure they can make money given their prices and production.

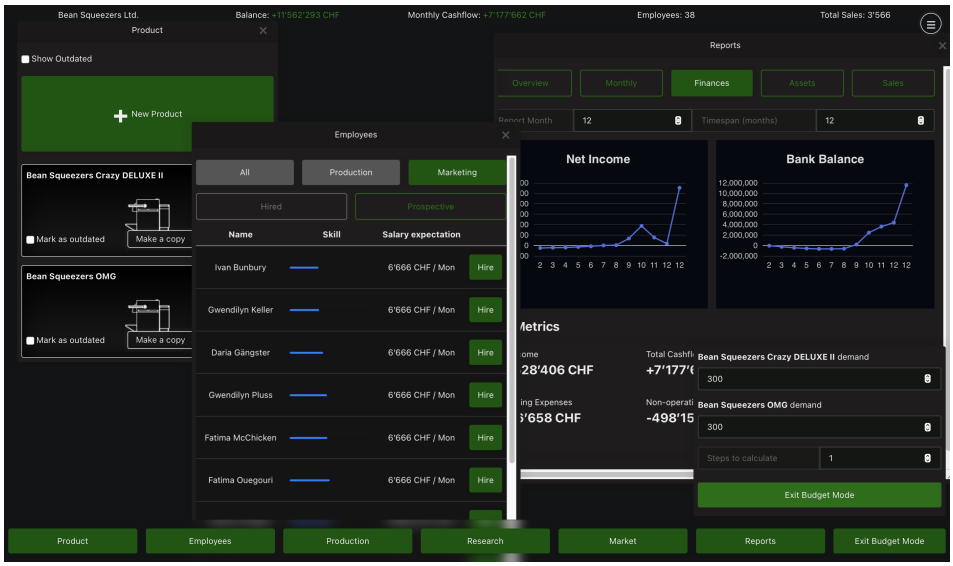

Figure 6: Prediction mode. The latest numbers shown in the graph represent the expected earnings.

4 Technology

When you start developing a piece of software of any kind, you have to pick a stack, that being: choosing which technologies you're going to build your project

on top of. This choice is probably the single most important decision of any project (right after choosing what to build); getting this wrong can lead to headaches for months to come.

This project experienced two major rewrites where I swapped platform! The back-end, where the *real calculations* take place is written in "Go" (commonly referred to as Golang), for those who aren't familiar with it, it's like if Python (Tygerjython) and C [2] had a baby; it has a simple syntax like python with 90% of the performance of C. The front-end (what the user actually sees) was originally written in HTML, CSS & JavaScript using "Wails" to connect the front- and back-ends. A few months into development I was experiencing too many issues with this setup: random crashes, tight coupling [3] and unexplainable bugs. This frustrated me to such an extent that I just stopped working on the project for multiple weeks. Eventually I decided to completely rewrite the front-end in Godot, a free game engine; the problem? I didn't know Godot. The smallest of changes required hours of reading the docs and struggling around.

In the end I came back to my senses and tried developing the front-end in JavaScript, this time learning Web-Sockets . As any developer knows, the 3rd time you write something is always better than the first two times. And so I stuck with this right until the end.

4.1 Implementation Details

The whole game is split into 4 modules:

- the Front-end,
- a webserver,
- the simulation,
- the game server
- and finally the orchestrator

This may seem complex at first glance, that's because it is. However, this strong separation of concerns has some quite positive emergent properties. Before I can get into those, we first have to understand what the concerns in question are.

(1) The first and most obvious concern is the simulation: the simulation is the core of the game however it doesn't stand on its own. The simulation expects clean,

[2] The "C" programming language is the basis of most modern software infrastructure, being used by every operating system. It's very performant, however it has very few features, making the developer have to develop many things from scratch

[3] Coupling in software refers to how when one part of the software is changed, another related part also has to be as well.

input all at once, which is antithetical to how humans work. (2) The next is that since this is a *multi-player* game, we need to handle input from multiple players at once. (3) Next up we there needs to be a way for the user to input data, also known as playing the game. (4) Finally needs to be something that co-ordinates all these processes.

The first two problems are solved with the game server. The game server connects to each client (the front-end) using Web-Sockets (We'll cover this later) and acts as a cache between the simulation and the front-end. The game server sanitises all the data from the simulation, removing unsupported values and saving decisions made on the front-end. For example when a player hires a new employee, the game server registers that the player did that. If their game crashes, they exit or any other issue, their changes are registered and when they rejoin they'll have the exact same state (and won't have to redo everything they had done previously). Eventually, when every player is ready, the server starts the simulation and simultaneously informs each client.

The 3rd problem is solved with the Front-end and webserver. The front-end provides a user interface for players to play the game through. Since the front-end is coded in Svelte [4], there also has to be a server to tell the browser what to display.

Finally, the orchestrator neatly packs everything into one single binary, handles errors of both servers and cleans up after you stopped playing.

4.1.1 Web-Sockets and Inter-process communication

One of the first non-trivial problems you experience when using multiple programming languages is how to send data from one to the other, or more generally how to send data from one program to another. This is when Inter-process communication (IPC) comes into play. Thanks to modern operating systems there are many ways to share data between programs, some of the most common are signals, sockets and pipes; all these systems are universally available and able to transfer data asynchronously, however not all are suited to all tasks. To understand which method to use, we need, we need to understand what they are.

The simplest IPC is probably the signal. Signals are sent through the operating system, interrupt the program at a certain point and run some function (by default exiting the program). What is done when a signal is received has to be registered by the recipient in advance. For example, when you you close a program you send the "kill" signal, the execution of the program is interrupted and the registered action is run; this typically entails saving unsaved data, cleanly stopping the program to make sure no data is corrupted. Signals are very easily implemented,

[4] Svelte is a front-end JavaScript and HTML framework, frameworks greatly simplify the process of developing a web-application

however their main downside is a deal-breaker for the game: the fact that they cannot transmit additional data other than their name.

Far more versatile are pipes. Pipes are buffers that one process can write to and the other can read from. Imagine someone sharing their screen on a call, the other person can see what's happening but can't edit what's happening. So for two process to have a two way communication channel, they need to pipes: one to the other program and another from it. The main advantage of pipes is that they are simple to implement, thanks to the fact that the operating system handles most of the hard work. Pipes are also often used by sys-admins to chain commands together: for example on MacOS or Linux you can get a sorted list of files in folder by entering `ls | sort`. Pipes are really useful tool to understand, however in this instance they are unsuitable because they only work on the local machine.

The most versatile are sockets, which use can use the network to receive data using internet protocols such as TCP or UDP. Sockets are like a letterbox in a large office: each employee has an assigned letterbox and can receive letters from the internal mail service or by post. These letterboxes are called ports and only a single process can own a given port but can receive from many processes. Using sockets a program can also receive data from other computers on the network: for example when you open a website, your browser sends a packet (letter) to the server on port 443, along with this packet there is a return port (eg. 31222), the server then replies with a packet containing the data that was requested sent to the port written in the request. This is the method used in Wisim. The great thing about sockets is that there is a huge amount of tools built on top of it, specifically for this project: Web-Sockets.

Web-Socket is a protocol built on top TCP that allows for the easy creation of a 2-way pipe between the server and a browser. They use a socket on both the client and server however they are able to bypass what I like to call the "port problem": namely that the server is (usually) not allowed to query the client [5]. It does this by having the client send an HTTP request to the server, the server then "upgrades" the request to a Web-Socket and instead of replying and closing the connection, it just leaves it open. Web-Sockets are by-far the easiest way to get a full-duplex (bi-direction) connection with a web-browser.

4.1.2 Implementing Web-Sockets

While Web-Sockets are very simple to implement, they have a totally different data-flow compared to HTTP: on the client you usually make an HTTP request and await a response, this is quite easy to implement, when you need data you ask for it and wait. With Web-Sockets you have to implement asynchronous event handling

[5] Typically the firewall on the client device will block all incoming connections unless they are replying to a client request.

because you never know when the server is going to send a new message.

On top of the previous, Web-Sockets is a binary format, that being that messages are not cleanly formatted like in HTTP with designated headers and status codes, instead they are just raw text, how the data is formatted is up to the developer. With me being *the developer*, that gives me a lot of room for stupid decisions. For example I spent a day inventing my own data encoding protocol which mimicked a command prompt. This might not sound all that dumb until you realise that both JavaScript and my Go back-end include functions for parsing JSON encoded messages.

4.1.3 WebAssebly

Since the game has some parts written in Go and some in JavaScript there will almost inevitably be some code that needs to be shared, lest I code 2 versions of the same function and *try very hard* to keep both versions in sync. For example the `CalculateProductStats` function which is used to (now this is going to surprise you) calculate the properties of products based on their components and the company. This function has to be identical on the front and back-end, otherwise players might get misled into thinking that their products are more or less good than they really are. Thankfully, other people have already had this problem and even came up with a solution. The solution, as you might guess by the section header, is called WebAssebly.

WebAssembly is a runtime for running non-JavaScript code on the browser. It is designed as a compile target for other language so that they can run in a standardised, architecture agnostic[6] way inside a browser.

Using WebAssembly along with some glue code [7], I was able to ensure that functions running on the frontend were *identical* to those running on the server.

5 Simulation

When attempting to simulate a consumer market, there are quite a few approaches you could choose between. The vast majority of models fall into 2 categories:

1. Statistical models: models where the simulation uses regressions and other statistical analysis tools to determine outcomes.

[6] Every CPU is based on a certain Architecture such as ARM or x86. Code built for ARM cannot run on x86 or vice-versa. What WebAssembly does is that it translates WebAssebly code to compatible machine code either just in time or ahead of time

[7] Since JavaScript is a dynamically typed language, there has to be some code that converts JS variables into statically typed Go variables

```
func calculateProductStatsWrapped() js.Func {
    f := js.FuncOf(func(this js.Value, args []js.Value) any {
        if len(args) != 2 {
            return fmt.Sprintf("Invalid arguments: Expected 2, Got %d",
                len(args))
        }

        err := checkArguments(args)
        if err != nil {
            return err.Error()
        }

        productStats, productionLineCost := simulation.
            CalculateProductStats(product, productCompontents)

        json, err := json.Marshal(struct {
            ProductStats       simulation.ProductStats
            ProductionLineCost float64
        }{productStats, productionLineCost})

        if err != nil {
            return err.Error()
        }

        return string(json)
    })

    return f
}
```

Figure 7: Glue code for the WebAssebly version of `CalculateProductStats`

2. Agent-based models: these models choose to, instead of simulating the whole market at once, instantiate individual *agents* that have the ability to independently make decisions.

There are 2 main advantages to statistical models: they are simpler to build, as you can often base them around simple regressions while still returning convincing results, and performance, unless the implementer makes grave mistakes, statistical models almost always outperform (in terms of speed) agent-based models of the same caliber.

For example, the cult classic city builder SimCity 4 was able to simulate regions

with populations up to 100 million with very minor performance issues, [8] while over 20 years later, games like Cities: Skylines 2 are still plagued by performance issues from simulating just 100'000 agents on modern multi-core hardware.

The biggest downside of statistical modelling for this project is flexibility. An agent-based model can simulate virtually anything, should the designer choose to, without necessarily major changes. This was the deciding factor for me, as the development of the game would take time, during which I would inevitably want to rework aspects of the simulation to optimise certain aspects.

5.1 How it works

The game works in 2 phases: the decisions phase and the purchasing phase. During the decisions phase players can analyise the data provided to them and ajust their companies, designing new products, hiring workers, purchasing production machines, etc. When every player is ready, the purchasing phase begins, during which the decisions made in the previous phase are executed.

5.1.1 Products

Players will develop many products during the course of the game. The products are made up of various comoonents, these comoonents along with the research levels of the company impact the properties of the product, for example using wooden construction will make you product slightly more ecological whereas using plastic will make it less so.

Players have to balance quality, ecology, durability and ethics with the manufacturing difficulty, design costs and price. This forces players to attempt to target a specific customer base lest they lose agaist someone who does. For example: the product that objectively has the best stats may be so expensive to produce that it has to be sold for multiple thousands of Francs, making it out of reach for most of the consumer base.

5.1.2 Production

Along with developing a product, companies of course have to produce them. To do so they need 3 things: production machines, employees and money. Every product has a so called "production cost", this refers to how difficult the product is

[8] This Kotaku article from 2014 shows off a city of over 100 million inhabitants, the Guinness world record, without any performance problems.
Link to article: kotaku.com/simcity-megacity-has-over-100-000-000-million-people-li-1629131314
World record: www.guinnessworldrecords.com/world-records/418977-largest-simcity-4-city-region

to produce; a production machine will produce more products with low production cost per month than products with a high cost all else being equal. Every machine has a specific "production capacity", as you might expect, this refers to how much "production" can be produced per month, thus how many of a specific product.

The second ingredient of production is employees, without them machines won't run. The key gameplay impact of employees is that as well as their salaries, the product of their skill and motivation determines how efficiently they work. If a machine has highly skilled, motivated workers it will produce even more than its listed production capacity. This encourages players to retain their best workers and treating them well. If they are over-worked they can burn out and become virtually useless. However, if you fire them, they might be snatched up by your competition, with them perhaps profiting off years of investment.

Last but not least is money. As with any physical good, they have to be made out of something and that something costs money. In fact during hard economic times the cost materials could increase dramatically [9], which, depending on the business strategy, can impact the company's profits.

5.1.3 Marketing

For people to buy a product, they have to know it exists. With marketing you can spread the word of your product and influence peoples' views on it at the same time. By investing large amounts in marketing quantity, you increase how many people see your product; later, when deciding which product to buy, customers only choose between the products they know. You can also influence how the product is perceived by configuring the marketing style. Marketing style along with marketing quality will positively impact how potential customers see the promoted properties of the product.

5.1.4 Consumers

The customer is simulated as a bundle of different characteristics, preferences, and needs; these all combine to produce unique behaviors for each individual agent. Taking inspiration from Economy Week, each customer has the following properties: [10]

[9] This feature, along with most other external factor related features, is not implemented

[10] The names provided the table may not be accurate to the corresponding implementations found in the code to simplify the explanation, as the implementation contains subtle differences for performance and technological reasons.

Property	Type	Description
Base Need	Number	The amount of products a customer wants. When their existing products wear out, they buy more.
Owned Products	List[number]	Keeps track of the remaining durability of owned products.
Known Products	List[number]	Keeps track of which products the customer knows about.
Purchasing Threshold	Number	How attractive a product has to be for the customer to buy it.
Savviness	Number	How well informed the customer is.
Loyalties	List[number]	How loyal the customer is to a certain company.
Loyalty Factor	Number	How much the customer's loyalties influence their
Preferences	Dictionary[number]	Which aspects of a product impact the customer's decisions the most. For example, a customer could value low prices more than they do the quality of the product.

The preferences of each customer is determined when the game is created using samples of a normal distribution for each of their preferences, these are then weighted and shifted based on results of playtesting. This means that for most properties most people don't necessarily care all that much however when looking at an individual, they might care a extremely about getting the best value product there is.

I planned to eventually implement a system of archetypes as described in Malcom Gladwell's The Tipping Point, such that each archetype has some preset preferences and others that are more muted like for example some extremely knowledgable people that recommend the best products to everyone else (like influencers), but this was cut due to time issues.

5.2 Purchasing phase

When all companies are ready, the internal workings of the companies are simulated and marketing impressions are calculated, each customer makes a decision on which product (if any) they buy.

Customers purchase the product they perceive is the best, that is the one that aligns the best with their preferences. This is calculated for each product using

the following formula[11].

$$\text{product opinion} = \begin{pmatrix} \text{product quality} \cdot \text{marketing style quality} \\ \dots \\ \text{product durability} \cdot \text{marketing style durability} \end{pmatrix} \tag{1}$$

$$\cdot \begin{pmatrix} \text{quality preference} \\ \dots \\ \text{durability preference} \end{pmatrix} \frac{\text{marketing quality}}{\text{customer savviness}} \tag{2}$$

Explaining the formula: The calculates the dot product of the product's properties and the marketing style (putting the focus on what was mentioned in the marketing). The dot product is used here as it can be used to calculate how "aligned" to vectors are; two orthogonal vectors will have a dot product of 0 while two collinear vectors will have a dot of their products. This is all then multiplied by how affected the customer is by the marking.

Finally the customer tries to buy as many of their chosen product as they need. If the product is not in stock, they don't buy anything.

5.3 An Example

There is a potential customer "John". The marketing of the products he has heard of gives John the following *perception* of the products:

Product	Quality	Ecology	Ethics	Durability	Price (CHF)
Bean Squeezer	12	11	5	3	1'000
Coffee Supreme	28	28	12	5	1'800
Eco Coffee 150	55	15	4	8	20'000

John has the following preferences:

- Quality: 1.4
- Ecology: 1.8
- Ethics: 2.0
- Durability: 0.4
- Price: 5.5

This means that his most important criteria is that the product is well priced. John also has a purchasing threshold of 80 meaning that he will only buy products that he feels align heavily with his preferences.

[11] Note: There might be slight differences in the implentationk. For simplicity's sake some edge cases and optimisation are not handled.

When John analyises all the products he knows about, he will check how well the products align with his preferences. When he looks at "Bean Squeezer" it aligns well with his preferences:

$$avrPrice = \frac{1'000 + 1'800 + 20'000}{3} = 7600$$

$$\begin{aligned} oppinion =& 12 \cdot 1.4 && (3)\\ &+ 11 \cdot 1.8 && (4)\\ &+ 5 \cdot 2 && (5)\\ &+ 3 \cdot 0.4 && (6)\\ &+ (0.5 + (avrPrice - 1'000)/avrPrice) \cdot 3.5 && (7)\\ =& 52.58 && (8) \end{aligned}$$

For each of the other products he has the following oppinions:

Product	Oppinion	Rank
Coffee Supreme	120.02	1
Eco Coffee 150	105.97	2
Bean Squeezer	52.58	3

John likes "Coffee Supreme" the most and so he tries to go and buy it, upon trying to buy it he realises it's out of stock so settles for the "Eco Coffee 150".

6 Tweaks and Balancing

While the core mechanics of the game might be perfectly functional, the game can only reach its full potential if there isn't just one winning strategy every time. This is where balancing comes into play, balancing is the act of tweaking the game so that certain gameplay styles are not too powerful or too weak.

6.1 Economic Balancing

In the Economic Simulation chapter I mentioned that the preferences of people is based on samples of a normal distribution along with some weights and shifts: this is where they come into play. The key aspect that was noticed in early play testing was that price had very little impact on purchases, this had mainly to do with sims'

too low "price" preference[12], so it was promptly raised. This led to another problem, namely that players couldn't produce products at that price without losing money. To fix that problem I gave companies more money to start off with (from 200'000 CHF to 800'000 CHF) and increased the productivity of their machines.

6.2 Research and Development

Research is one of the hardest mechanics to optimise while keeping the game fun. The issue is that research having too high an effectiveness would defeat the need to ever meaningfully change anything in your company, on the other side, if research is too weak then what's the point? The solution I found to this problem was to slightly lower the effectiveness of research and only apply it new products, this way players are forced to develop new products and maybe try out some different things compared to what they had before.

7 What I Learned

As with any software project, there are few things that go to plan and many lessons that were learnt along the way. One of the key lessons I learnt during the development of this project is to avoid *premature optimisation*.

7.1 Premature Optimisation

> Premature optimization is the root of all evil (or at least most of it) in programming.
>
> **Donald Knuth**

"Premature optimisation is the act of devoting a significant amount of optimising for task you might not truly need to optimise for."

A funny example of this was while developing budget mode. In budget mode the player is able to see the predicted outcome of the decisions they are about to make. To be able to clearly differentiate between budget mode and normal mode, I wanted the colour scheme to change such that instead of blue being the primary colour, green was.

[12] This was also due to a bug I only noticed while writing this section that severely increased the population's tolerance for high prices

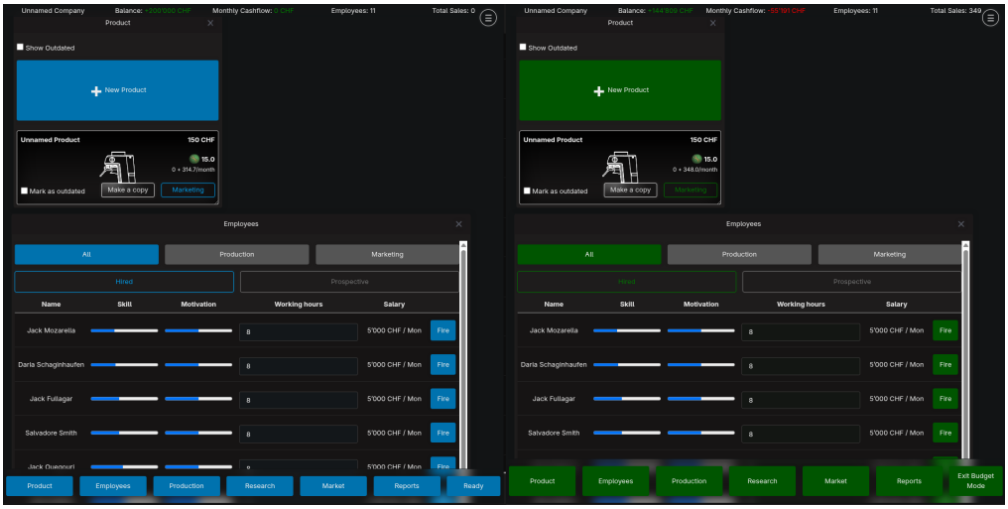

Figure 8: What I wanted to achieve

Having heard of relative colours using the CSS[13] `color()` and `color-mix` functions I wanted to adapt my current style sheet to use these relative colours. I just had one problem: the style sheet I was using was multiple hundreds of lines long (quite long!) and I didn't want to manually edit all of that! So what is the logical answer to this predicament? Is it to just make a new style sheet that approximated the previous one except that it had these relative colours, given that I was using a very small subset of the features of the style sheet? NO! The answer is to write a small program that reads, parses and extracts colours from a CSS file, converts the colours to relative colours (if possible) and finally rewrites the new CSS file. This, in fact, was not the most logical solution. It is not only unnecessarily trying to automate a task that realistically takes a half hour, but it also steps into the *extremely complex*[14] domain of digital colours, a domain that I had almost no knowledge in.

I ended up wasting a week of time calculating and coding this "simple" app to convert colours. The maths to derive the relative components of a colour already took multiple pages and the program itself was hundreds of lines long. This misadventure thoroughly taught me not to try to prematurely optimise something that, in the end, I didn't even need.

7.2 Feature Creep

[13] CSS is the technology used to style websites and web apps. CSS files are also referred to as style sheets.

[14] For more info on digital colours see: "Everything About Color in 25 Minutes" by Juxtopposed. https://www.youtube.com/watch?v=srRI7yMjGz0&t=6s